

MediConnect: o platforma web pentru telemedicina cu consultatii video si plati in escrow

Bosna Marinel¹ and Conf. Dr. Daniel Pop¹

¹Facultatea de Informatica, Universitatea de Vest din Timisoara

Abstract

MediConnect is a web platform that brings together online appointments, video consultations, digital prescriptions and payments held in escrow until the visit is confirmed. The paper describes the architecture chosen to support these features: a compact Spring Boot backend organized around clearly separated functional areas, paired with a React frontend, in which business rules are kept apart from technical details and external providers (video, payments, identity) are integrated through interchangeable adapters. The key idea is that the two critical moments of a consultation - the visit itself and the release of funds - are driven by real external events instead of user clicks in the interface, which keeps the system consistent even when something goes wrong. The following sections present these design decisions, the challenges encountered while building the prototype and the criteria that will be used to evaluate the platform going forward.

1. INTRODUCERE SI MOTIVATIE

Problema centrala abordata in MediConnect nu este absenta solutiilor de telemedicina, ci faptul ca parcurusul unei consultatii ramane fragmentat in pasi independenti, lipiti intre ei prin coordonare manuala. In practica, un pacient roman suna pentru a rezerva slotul, achita la cabinet sau prin transfer, intra in apel printr-un alt serviciu si, eventual, primeste reteta pe email sau pe hartie. Cand un pas esueaza, corelarea ramane in sarcina pacientului si a personalului clinicii: rambursari procesate manual, programari replanificate prin telefon, retete pierdute intre sisteme. Costul real nu este vizibil intr-un raport tehnic, ci in timp pierdut, ne incredere si consultatii abandonate inainte sa inceapa.

Lucrarea pleaca de la ipoteza ca aceste pasi pot fi consolidati intr-o singura aplicatie web, in care pacientul gaseste un medic, rezerva, plateste in siguranta, intra in consultatie si primeste reteta intr-un flux continuu. Doua constrangeri ghideaza arhitectura: datele medicale sunt categorie speciala in sensul GDPR si cer protectie suplimentara, iar plata in avans nu poate fi virata medicului inainte ca platforma sa fie sigura ca s-a desfasurat consultatia. Aceste doua principii - confidentialitate stricta si plata cu retinere temporara, numita escrow - sunt firul rosu care leaga toate sectiunile lucrarii.

Directia tehnica propusa este o arhitectura monolitica modulara, in care logica de business si modulele de domeniu sunt protejate fata de detaliile de framework, iar furnizorii externi devin adaptoare in-

terschimbabile. Aceasta directie pastreaza vitezele de dezvoltare specifice unui proiect Spring MVC, dar introduce disciplina necesara pentru ca aplicatia sa ramana mentenabila pe masura ce se adauga noi tipuri de consultatii, noi metode de plata sau noi furnizori video.

2. ARHITECTURA SISTEMULUI SI DISTRIBUTIA RESPONSABILITATILOR

MediConnect este organizat pe trei niveluri colaborative. La nivelul superior se afla actorii umani: pacientul, medicul si managerul de clinica, fiecare cu propriul ecran, propriile permisiuni si propriul ritm de interactiune cu platforma. Aceeasi aplicatie React serveste toate cele trei roluri, dar feature-urile sunt grupate pe folder-e dedicate, iar rutele protejate sunt rezolvate pe baza claim-urilor din token-ul Keycloak.

La nivelul mijlociu, frontend-ul React 19 cu Vite si TypeScript expune o experienta SPA in care fiecare modul functional (Programari, Consultatie video, Plati, Notite medicale) este izolat ca feature folder, cu propriile hook-uri TanStack Query pentru server state si propriile componente. Frontend-ul nu detine logica de business; el orchestreaza interactiunea cu API-ul si gestioneaza tranzitia intre ecrane.

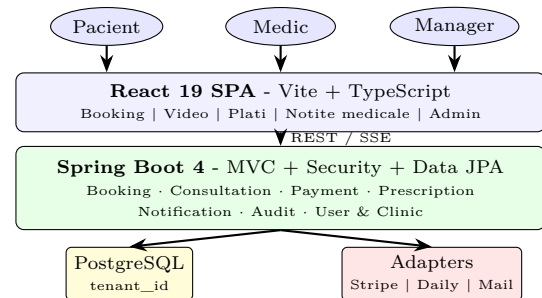


Figura 1: Distributia pe actori, frontend si backend: o singura aplicatie React serveste trei roluri, un singur monolit Spring deserveste mai multe bounded contexts.

La nivelul inferior, backend-ul Spring Boot pastreaza toata logica de business, validările si stările critice. Modulele sunt grupate pe bounded contexts si comunica prin servicii interne sau evenimente in proces, niciodata prin acces direct la entitățile altui context. Aceasta limita este aplicata fizic prin pachete cu visibility restrans si verificata in build prin reguli ArchUnit.

3. MONOLIT MODULAR CU ARHITECTURA LAYERED HYBRID

Pentru o aplicatie web cu un singur deploy initial, varianta naturala este Spring MVC pe trei

niveluri: Controller, Service, Repository. MediConnect porneste din aceasta structura, dar adauga deliberat un layer de domeniu, plasat sub Service, in care traiesc regulile cu adevarat importante: o programare nu poate fi mutata daca a inceput deja consultatia, o plata nu poate ajunge in HELD fara un eveniment Stripe corespunzator, iar o reteta nu poate fi semnata electronic de un cont care nu este verificat. Aceste reguli sunt exprimate ca metode pe entitatile de domeniu, nu ca if-uri imprastiate prin Service. Astfel, layer-ul Service devine subtil: el orchestreaza, valideaza autorizarea si delega catre domeniu, fara a duplica regulile.

Al doilea element imprumutat din Clean Architecture este inversarea dependintelor pe granita dintre Application si Infrastructure. Serviciul de consultatie nu cunoaste `com.stripe` sau `co.daily`; el lucreaza cu interfețe locale precum `PaymentGateway`, `VideoProvider` si `Notifier`, implementate de adaptoare din pachetul de infrastructura. Aceasta organizare permite injectarea de adaptoare in-memory in testele unitare si izoleaza schimbarile de SDK la un singur fisier; un upgrade al unui SDK extern nu mai atinge logica de business.

A treia trasatura este existenta unei limite explicite pentru tranzactii. Operatiile critice (rezervare slot, capturare plata, eliberare fonduri) sunt declansate dintr-un singur use case si ruleaza intr-o tranzactie Spring sincrona; comunicarea cu serviciile externe se face fie inainte de a deschide tranzactia, fie dupa commit, prin Spring Events sau prin job-uri Quartz, niciodata in mijlocul unei tranzactii deschise. Astfel se evita situatiile in care apelul catre Stripe sau Daily ramane blocat in fundal in timp ce conexiunea la baza de date este tinuta deschisa, problema clasica a aplicatiilor care amesteca I/O extern cu persistenta.

4. INTEGRAREA SERVICIILOR EXTERNE ORIENTATA PE SERVICII

Daca interiorul aplicatiei este monolit, marginea ei este in mod intentionat service-oriented. Profilul tehnic al unei sesiuni video difera radical de cel al unei plati sau al unui email tranzactional, iar tratarea acestor componente ca module simple in core ar lega productia de un anumit furnizor si ar transforma coreul intr-un proxy pentru SDK-uri.

Pentru fiecare componenta separata exista un contract intern stabil. `VideoProvider` expune metodele `createRoom`, `issueMeetingToken` si `getRecordingUrl`; `PaymentGateway` expune `reserveFunds`, `captureHold`, `releaseToConnect` si `refund`. Inlocuirea unui furnizor (de exemplu Daily.co cu Twilio Video) inseamna scrierea unui nou adaptor si modificarea unei singure linii in configurare, niciodata atingerea logicii de business.

Avantajul cel mai important al acestei organizari este faptul ca starea financiara si starea consultatiei avanseaza in mod controlat, fiecare la momentul ei. Cererea HTTP a pacientului produce doar starea `RESERVED`; trecerea in `HELD` apartine

webhook-ului `payment_intent.succeeded`, iar trecerea in `RELEASED` apartine unui job Quartz care evalueaza fereastra de 48 de ore. Aceasta separare elimina o categorie intreaga de incidente cauzate de timeout-uri sau dublarea cererilor in UI si face fluxul reproductibil in teste: acelasi webhook trimis de doua ori produce acelasi rezultat, iar absenta unui webhook este detectata de un job de reconciliere care ruleaza periodic.

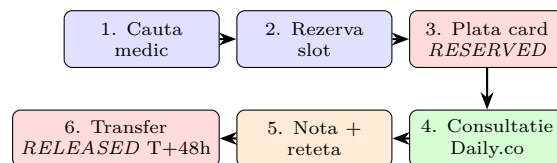


Figura 2: Parcursul unei consultatii MediConnect, de la cautarea medicului pana la eliberarea fondurilor din escrow: actiunile pacientului declanseaza doar starile initiale, restul tranzitiilor fiind conduse de evenimente externe si de job-uri Quartz.

5. MULTI-TENANCY, IZOLARE SI SECURITATE

MediConnect este conceput pentru a deservi simultan mai multe clinici si medici independenti pe aceeași instanță. Fiecare entitate organizationala devine un tenant, iar fiecare programare, consultatie, plata si nota medicala poarta in mod obligatoriu coloana `tenant_id`. Strategia aleasa pentru prototip este shared database, shared schema cu izolare logica, deoarece versiunea actuala nu justifica overhead-ul unei abordari database-per-tenant; trecerea catre schema-per-tenant ramane posibila exact pentru ca `tenant_id` este explicit in toate repository-urile.

Disciplina de izolare este aplicata in trei locuri: filtrul de securitate extrage clinica si rolurile din tokenul de autentificare si le ataseaza fiecarei cereri, accesul la date aplica automat acest filtru, iar mesajele venite de la furnizorii externi rezolva apartenenta exclusiv prin identificatorul lor de cont, fara a accepta cereri din browser. Notele medicale sunt criptate la stocare, iar cheile sunt gestionate intr-un manager de secrete extern, astfel incat o copie a bazei de date nu expune continutul consultatiilor.

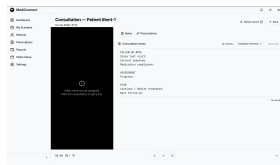
Pe partea de autorizare, MediConnect foloseste un model de permisiuni fine peste roluri (de exemplu anulara propriei programari, anulara oricarei programari din clinica, rambursarea unei plati), astfel incat aceeasi actiune poate fi permisa pacientului doar asupra propriilor date si managerului asupra intregii clinici, fara a duplica logica in cod.

6. STADIUL CURENT SI TESTABILITATE

Prototipul actual implementeaza intregul parcurs pentru o clinica: pacientul se autentifica, rezerva un slot, plateste cu un card de test, intra in consultatia video, primeste reteta in panoul propriu si vede banii in starea "retinuti", eliberati automat dupa fereastra de 48 de ore. Pe partea medicului exista un spatiu de lucru cu calendar, panou de notite cu salvare opti-

mista si listare a retetelor emise. Fluxul este suficient de complet pentru a demonstra ca pacientul nu mai are nevoie sa coordoneze manual etapele si ca personalul clinicii nu mai gestioneaza in paralel rambursari si replanificari.

Validarea testabilitatii a fost confirmata prin trei tipuri de teste. Testele unitare ale serviciilor de business folosesc adaptoare in-memory pentru `PaymentGateway` si `VideoProvider` si ruleaza in cateva sute de milisecunde fara a porni context-ul Spring. Testele de integrare ale stratului de persistenta folosesc `Testcontainers` cu un PostgreSQL real pentru a valida specificatiile cu `tenant_id`. Testele de webhook trimit acelasi `event_id` de doua ori si verifica idempotentia prin ledger-ul `webhook_events`.



7. LUCRARI CONEXE SI POZITIONARE

In zona platformelor comerciale, `DocPlanner` (`Docbook` in Romania) si `Doctolib` acopera bine programarea online si parcursul pacientului in afara cabinetului, dar lasa consultatia video si plata ca module externe; `Teladoc Health` include consultatia video, insa fluxul de plata este orientat pe asigurari, nu pe escrow individual. In literatura academica, studiile despre arhitecturi de telemedicina trateaza preponderent securitatea `WebRTC` si conformitatea `HIPAA` si nu propun un model integrat care sa cuprinda si fluxul financiar.

Fata de aceste abordari, `MediConnect` alege intentionat un punct intermediar. Un monolit clasic in stil Spring MVC ar fi simplu de pornit, dar ar concentra toate apelurile catre SDK-uri in Service si ar face testarea izolata greoaie. O arhitectura microservicii ar oferi independenta deplina, dar ar introduce comunicare distribuita, versionare API, consistenta eventualizata si pluralitate de deploy-uri, fara un castig real cand multe operatii apartin aceleiasi consultatii. Versiunea propusa pastreaza forma layered ca scheletul aplicatiei si preia din Clean Architecture doar elementele care aduc valoare concreta: domeniul activ, porturile interne si testabilitatea.

8. PROVOCARI SI CRITERII DE EVALUARE

Pe parcursul prototipului au aparut trei intrebari practice la care platforma trebuie sa stie sa raspunda singura. Prima: cand trecem banii la medic? Prea repede, pacientul nu mai apuca sa reclame daca ceva nu a fost in regula; prea tarziu, medicul ramane fara plata. Am ales o fereastră de 48 de ore, scurta pentru medic si suficienta pentru pacient. A doua: ce facem cand ceva merge prost - pacientul nu apare, internetul pica, cineva renunta inainte de start? In loc sa decida cineva de la clinica la fiecare incident, platforma vede ce s-a intamplat in apel si reactioneaza singura. A treia: cum protejam datele cand mai multe clinici folosesc aceeași platforma? Fiecare clinica vede

doar pacientii sai, iar notele consultatiei raman ilizibile chiar daca cineva ar obtine o copie a bazei.

Coerenta arhitecturii este analizata in continuare prin criterii non-functionale:

- **Modularitate:** fiecare bounded context are pachet propriu, fara acces direct la entitatile celorlalte.
- **Testabilitate:** serviciile pot rula offline cu adaptoare fake si fara context Spring.
- **Izolare:** accesul la date este filtrat prin `tenant_id`, rol si permisiuni fine.
- **Extensibilitate:** un nou furnizor video sau de plati se adauga prin implementarea unui port intern, nu prin atingerea logicii de business.
- **Simplitate operationala:** core-ul ramane un singur deploy, monitorizat printr-un singur set de metrice.

9. CONCLUZII SI PASII URMATORI

Lucrarea descrie cum o aplicatie web construita pe Spring Boot si React poate consolida programarea, consultatia video si plata in escrow intr-un singur monolit modular, pastrand totodata disciplina arhitecturala adusa de Clean Architecture. Decizia centrala - cuplarea a doua masini de stare paralele prin evenimente externe - rezolva categoria de incidente in care plata si consultatia avanseaza din pasi diferiti si ar trebui sa fie corelate manual.

Pe termen scurt, intentia este finalizarea jobului Quartz de detectie no-show, completarea fluxului de dispute, adaugarea Server-Sent Events pentru indicatorul de prezenta online a medicului si scrierea unei suite Playwright pentru parcursul pacient-medic. Tot in aceasta etapa intra externalizarea stocarii documentelor dintr-o consultatie (note semnate, rețete in format PDF, atasamente trimise de pacient) catre AWS S3 cu bucket-uri separate pe tenant, cifrare SSE-KMS si URL-uri pre-semnate cu durata scurta de viata, astfel incat backend-ul Spring sa nu mai serveasca direct continut binar.

Pe termen mediu, `MediConnect` urmeaza sa fie deployat intr-o configuratie cloud reproductibila: containerizare cu Docker, orchestrare pe AWS ECS sau EKS, baza de date gestionata prin RDS, secret-uri in AWS Secrets Manager si un pipeline CI/CD care ruleaza testele unitare, testele `Testcontainers` si testele Playwright inainte de promovarea unei imagini in productie. Tot pe termen mediu se intentioneaza introducerea unui asistent AI dedicat medicului, care, pornind de la notele consultatiei si de la istoricul pacientului, sa propuna ipoteze de diagnostic, sa sugereze investigatii suplimentare si sa pre-completeze structura rețetei; modelul va fi integrat ca un adaptor in spatele unei interfete `ClinicalAssistant`, similar cu `VideoProvider` si `PaymentGateway`, astfel incat schimbarea furnizorului de model sa nu afecteze logica de business si confidentialitatea datelor sa fie controlata explicit.